



LECTURE 12

IMPLEMENTATION: KNOWLEDGE FUSION BY REWIRING ATTENTION

CODE WALKTHROUGH

DATASCI290: NEUROSymbOLIC AI

FEBRUARY 18 2026



The notebook demonstrates the key steps in the core attention rewiring pipeline

- A tiny knowledge graph: a few (head, relation, tail) triples
- A question → rank & select top-k triples
- Convert selected triples into per-layer memory (K,V) tensors
- **Rewire attention:** prepend memory K,V to normal K,V in chosen layers
- Compare: baseline vs prompt-stuffing vs rewired attention



Example Knowledge Graph

Super SMALL !We deliberately use a tiny set of triples

- Triples:
 - ("ICML_2025", "held_in", "Dubai")
 - ("Dubai", "located_in", "United Arab Emirates")
 - ("WWW_2025", "held_in", "Sydney")
- Questions: "Which country was ICML_2025 held in?"

The Algorithm

- 1) Rank triples for the question
- 2) Select top-k + normalize weights
- 3) Build per-layer memory K,V from triple texts
- 4) Patch attention modules
 - (swap self_attn $\rightarrow$ wrapper)
- 5) Enable KGA for a layer window; generate; compare outputs

Algorithm 1 Process of language models equipped with KGA

Require: Candidate triples $\mathcal{G}$, Query X , temperature τ .

Ensure: Model predictions $\mathcal{M}_{\Theta}(X)$ based on our KGA.

```
1: // Initialize representations of  $Z \in \mathcal{G}$ ,  $X$  from token embeddings
2:  $Z^{(0)}, X^{(0)} \leftarrow \text{TokenizeAndEmbed}(Z, X)$ 
3: // Model Forward Pass
4: for  $layer\_idx$  in  $\text{enumerate}(\text{hidden\_layers})$  do
5:   // 1. Estimate triple relevance
6:   // 1.1. Extract valuable clues
7:   Estimate  $C$  via Eq. 3 based on  $X$  and  $Z$ 
8:   // 1.2 Perform forward pass of triples
9:    $Z^{(l+1)} \leftarrow \text{KGA\_layer}(Z^{(l)})$ 
10:  // 1.3 Consolidate clues and compute relevance
11:  Consolidate and perform  $\alpha_Z$  estimation via Eq. 4 - Eq. 6
12:  // 2 Adaptive integration
13:  Perform knowledge fusion via Eq.2
14:   $X^{(l+1)} \leftarrow \text{KGA\_layer}(X^{(l)})$ 
15: end for
16: Logits  $\leftarrow \text{LMHead}(X^{(L)})$ 
17: return Decode(Logits)
```

Function: select_topk_triples()

From scores → chosen triples + weights

```
def select_topk_triples(question, triples, k=2):  
    ranked = rank_triples(question, triples)  
    selected = [t for s, t in ranked[:k]]  
    scores = torch.tensor([max(s, 1e-6) for s, _ in ranked[:k]])  
    weights = scores / scores.sum()  
    return selected, weights
```

- rank_triples() can be as simple as token overlap (demo) or embeddings (better).
- weights sum to 1, later used to scale memory contribution (we scale V).

Triples → Text Tokens

We encode structure as a short string

```
Trip = Tuple[str, str, str]

def triple_to_text(t: Trip) -> str:
    h, r, o = t
    return f"({h} , {r} , {o} )"
```

- Why text? It lets us reuse the model's own tokenizer + embeddings.
- Later we convert these tokens into attention-native memory (K,V).

What is “memory K,V”?

Attention reads from Values via similarity to Keys

- In self-attention, each layer builds Q,K,V from token hidden states.
- We **precompute extra K,V** for triple tokens → “memory tokens”.
- Then we **prepend memory K,V to the normal K,V** so the prompt can attend to them.

```
# Per layer memory shapes
K_mem, V_mem: [1, kv_heads, M, head_dim]
# Later expanded to batch and concatenated on the sequence axis
```

Function: TripleMemory container

Stores memory K,V for every layer

```
@dataclass
class TripleMemory:
    kv_per_layer: List[Tuple[Tensor, Tensor]] # (K,V) per layer
    # each: K,V: [1, kvh, M, D]
    valid_mask: Tensor # [1, M] bool
```

- `kv_per_layer[l]` gives the memory K,V to inject into layer `l`.
- `valid_mask` marks which memory token slots are real vs padding.

Function: `_encode_single()`

Tokenize each triple text to fixed length

```
def _encode_single(text, max_tokens=32):  
    enc = tokenizer(  
        text, return_tensors="pt",  
        truncation=True, max_length=max_tokens,  
        padding="max_length"  
    )  
    return enc["input_ids"], enc["attention_mask"]
```

- We pad every triple to `max_tokens` so shapes are uniform.
- `attention_mask=1` for real tokens, 0 for padding → used to zero out pads.



Function: `build_triple_memory()`

Core: per-layer K, V from triple hidden states

- Step A: run triple tokens through `model.model` with `output_hidden_states=True`
- Step B: for each layer ℓ , apply that layer's `k_proj` and `v_proj`
- Step C: concatenate all triples' K, V along the token axis $\rightarrow$ memory length M
- Step D: store (K, V) per layer in `TripleMemory`

build_triple_memory(): get hidden states

We need token representations at every layer

```
out = model.model(
    input_ids=ids,
    attention_mask=am,
    output_hidden_states=True,
    use_cache=False,
    return_dict=True,
)
all_hidden.append(out.hidden_states)  # (L+1) tensors [1,T,H]
all_valid.append(am[0].bool())       # [T]
```

- `hidden_states[ℓ]` is the token vectors after layer ℓ (shape $[1,T,H]$).
- We will project those vectors into the layer's K/V head space.

build_triple_memory(): per-layer K/V projection

Same projection matrices that the original model uses in attention

```
attn = layers[layer_idx].self_attn
kv_heads = attn.num_key_value_heads (or config)
head_dim = attn.k_proj.out_features // kv_heads

h = all_hidden[j][layer_idx] # [1,T,H]
k = attn.k_proj(h).view(1,-1,kv_heads,head_dim).transpose(1,2)
v = attn.v_proj(h).view(1,-1,kv_heads,head_dim).transpose(1,2)
# k,v: [1,kvh,T,D]
```

- We do this for each triple j, then concatenate along token axis.

build_triple_memory(): pads + weights

Mask out pads; scale triple contribution

```
k = k * am[:, None, :, :]  
v = v * am[:, None, :, :]  
# scale V by relevance weight  
v = v * w[j]  
  
K = torch.cat(ks, dim=2) # [1,kvh,M,D]  
V = torch.cat(vs, dim=2)  
valid_mask = cat(all_valid).view(1, -1) # [1,M]
```

- Padding tokens are zeroed so they don't add noise.
- Weights make “more relevant” triples contribute more strongly.

The Attention Rewiring

We replace each layer's self_attn with a wrapper

- Baseline path: wrapper delegates to original (HF) attention module.
- KGA path: wrapper computes attention itself and prepends memory K,V.
- The “rewire” is simply: `layer.self_attn = KGAWrappedAttention(original_attn, ...)`

```
# Key rewiring line
layer.self_attn = KGAWrappedAttention(base_attn=layer.self_attn, ...)
```

KGAWrappedAttention.forward(): baseline path

If kga_enabled=False, behave like the original model

```
if not self.kga_enabled:
    out = self.base_attn(
        hidden_states=hidden_states,
        attention_mask=attention_mask,
        position_ids=position_ids,
        ...
    )
    return self._return_two(out)
```

- This is why baseline + prompt-stuffing stay stable after rewiring.
- `_return_two()` trims HF return variants to exactly (output, attn_or_None).

KGAWrappedAttention.forward(): KGA path (core)

Compute Q/K/V, prepend memory, extend mask, run attention

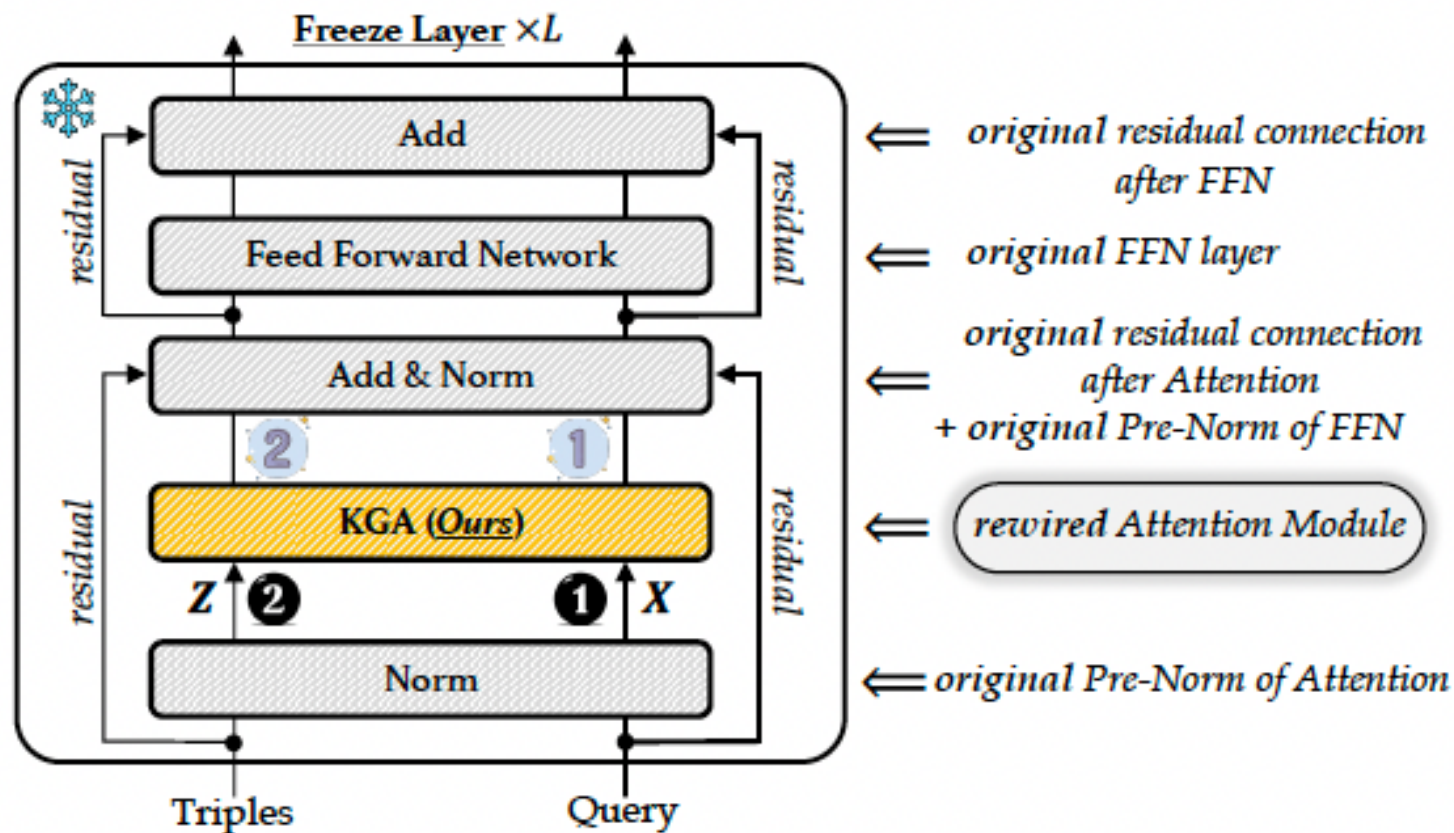
```
# 1) Q/K/V projections
Q = q_proj(hidden_states)
K = k_proj(hidden_states)
V = v_proj(hidden_states)

# 2) Prepend memory (per layer)
mem_k, mem_v = triple_memory.kv_per_layer[layer_idx]
K = cat([mem_k, K], dim=seq)
V = cat([mem_v, V], dim=seq)

# 3) Extend attention mask
mask = cat([mem_mask, orig_mask], dim=key_len)

# 4) Attention
P = softmax(Q K^T / sqrt(d) + mask)
out = P V
```

- This is the “fusion” step: prompt tokens can attend into triple memory.



Function: patch_llama_attention()

The loop that swaps modules layer-by-layer

```
originals = []
wrappers = []
for i, layer in enumerate(model.model.layers):
    base = layer.self_attn
    originals.append(base)
    wrapped = KGAWrappedAttention(
        base_attn=base,
        triple_memory=triple_memory,
        layer_idx=i,
        model_config=model.config,
        apply_rotary_pos_emb=apply_rope,
    )
    layer.self_attn = wrapped
    wrappers.append(wrapped)
model._kga_original_self_attn = originals
return wrappers
```

- This is the literal “rewire”: `layer.self_attn` gets replaced.
- We keep originals so `unpatch_llama_attention()` can restore them.

Enabling Select Layer Window

Turn KGA on for some layers, keep baseline intact

```
def set_kga_layer_window(attn_modules, start, end):  
    for i, m in enumerate(attn_modules):  
        m.kga_enabled = (start <= i < end)  
  
# Baseline:  
set_kga_layer_window(attn_modules, 0, 0)  
# KGA (middle third):  
L = len(attn_modules)  
set_kga_layer_window(attn_modules, L//3, 2*L//3)
```

- We precompute memory for all layers, but inject only in chosen layers.
- Reason: stability + better “where to use knowledge” behavior.

Comparison

Same question, three ways of using triples

```
print('--- Baseline ---')
set_kga_layer_window(attn_modules, 0, 0)
print(generate_text(question))

print('--- Prompt stuffing ---')
print(generate_text(prompt_stuff(question, selected)))

print('--- KGA fusion ---')
set_kga_layer_window(attn_modules, L//3, 2*L//3)
print(generate_text(question))
```

- Baseline: no knowledge provided.
- Prompt stuffing: triples are visible tokens in the prompt.
- KGA fusion: triples become attention memory (not visible text).

TO SCALE

- The notebook demonstrates the **core algorithmic idea** of KGA: *structured knowledge becomes attention-visible memory*.
- **For an at-scale solution**
 - **Retrieval and ranking must become robust and scalable:** We used a handful of hand-selected triples. At scale, you would retrieve from thousands or millions of triples using embedding search or graph traversal. You must then score and rerank candidates carefully, eliminate distractors, detect contradictions, and possibly compose multi-hop chains. Poor retrieval will silently degrade performance.
 - **Memory structure must be preserved explicitly:** Here triples are converted into plain text tokens. In a full system, you would preserve head–relation–tail boundaries, track which memory tokens belong to which triple, and potentially maintain entity-aware grouping.
 - **Memory influence must be calibrated:** Memory competes with prompt tokens inside attention with minimal control. In practice, you need explicit gating or scaling so memory helps without destabilizing generation. This may include per-layer mixing coefficients, per-triple weights, or even light finetuning to learn how strongly memory should influence attention.
 - **Masking and positional handling must be precise** Injecting memory tokens requires carefully extending attention masks and ensuring positional encodings (e.g., RoPE) remain consistent.
 - **Model portability must be handled cleanly.** Different model families implement attention differently (GQA vs full MHA, different return signatures, cache types, chat formatting). A production KGA implementation must abstract over these variations without breaking generation.



TEASER QUESTION

HOW ABOUT “WIRING IN” DATALOG RULES IN A SIMILAR WAY ??



DATALOG FUSION MUSINGS

■ What *could* work

- Encode rules as structured memory units
 - Head :- Body1, Body2
 - Store body predicates and head predicate as segmented memory tokens
- Inject rule memory (K,V) just like triples
 - Let question tokens attend to rule bodies
 - Let derived entities attend to rule heads
- Use multi-layer injection to simulate multi-hop reasoning
 - Early layers: match facts to rule bodies
 - Mid layers: combine matched conditions
 - Later layers: bias toward rule head conclusion

DATALOG FUSION MUSINGS

- Why this is harder than triples
 - Rules are *conditional*, not atomic facts
 - Need structured binding between variables (X, Y, Z)
 - Attention is soft weighting, not discrete logical firing
 - “Partial satisfaction” is not true logical entailment
 - Variable unification is not naturally represented by token similarity
- What will likely not work directly
 - Negation ($\text{not } p(X)$)
 - Attention cannot enforce absence of evidence
 - Softmax cannot express logical exclusion cleanly
 - Recursion with fixed-point guarantees
 - Transformer layers are finite depth
 - No guarantee of convergence like Datalog evaluation
- Tensor-Logic: <https://tensor-logic.org>